Index for Text Retrieval

Multimedia Retrieval · Chapter 4

Multimedia Retrieval - 2026 - Uni Basel

Roger Weber

Contents

4 - Index for Text Retrieval **1**
- Inverted Files 2
  - Why scanning does not scale 2
  - Inverting the layout 2
  - The structure of the index 3
  - Boolean retrieval over postings 4
  - From matching to ranking 5
- Ranked Retrieval over Inverted Files 6
  - Retriever and ranker 6
  - Two evaluation strategies 6
  - The three models over postings 7
- Filtering, Facets, and Compression 9
  - Filtering on metadata 9
  - Faceted search 9
  - Compressing postings lists 9
  - Metadata alongside postings 10
- Practical Frameworks 11
  - Full-text search in your database 11
  - Lucene 12
  - Higher-level pipelines 15
- Scaling Out 16
  - The distributed engines 16
  - Parallelism within a node 16
  - Sharding across nodes 16
  - Replication and availability 17
  - Distributing across regions 18
- Summary 19
  - Approach Comparison 19
  - Key Takeaways 19
  - Key Formulas 19
  - Self-Check Questions 19
  - Further Reading 20

4 - Index for Text Retrieval

By now we have most of the pieces of a text retrieval system. We can turn documents and queries into terms through tokenization and normalization, and we can say what a good result looks like using precision and recall. Chapter 1 gave us models that score how well a document matches a query, from Boolean matching to the vector space model and BM25. One practical question remains: how do we find the matching documents quickly when the collection holds millions of them? Scoring every document against every query, one after another, does not scale.

Fast retrieval is a balancing act between three competing goals: the quality of the ranked results, the latency of each query, and the storage the index consumes. The inverted index is the data structure that has held this balance for decades. It made full-text search practical on the modest hardware of the 1970s and 1980s, and it still sits at the core of today's search engines. Its defining property is that the work per query grows with the number of query terms, not with the size of the collection. Only recently have dense vector methods reached comparable speed at large scale, and even then they are often paired with an inverted index rather than replacing it. We return to dense retrieval in later chapters.

This chapter opens up the mechanics. We start from a small example and build the inverted file, then show how the same structure serves the retrieval models from Chapter 1. From there we add metadata filters and faceted search, and we look at how compression keeps the index small. We close with practical, low-cost ways to run production-grade search: full-text search inside a database you may already operate, the Lucene library, and the distributed engines built on top of it.

Inverted Files

Consider a tiny collection of three documents:

- D_1: “The cat sat on the mat”
- D_2: “The dog chased the cat”
- D_3: “The cat and the dog played”

A query for “cat” should return D_1, D_2, and D_3; a query for “dog” should return D_2 and D_3. With three documents we could simply read each one and check. With three million documents we cannot. We need a structure that takes us straight from a query term to the documents that contain it, without touching the rest of the collection. That structure is the inverted index, and this section explains why it makes query cost depend on the query rather than on the size of the collection.

Why scanning does not scale

Recall from Classical Text Retrieval that a document is represented as a sparse vector over the vocabulary. Assume a collection of N documents and a vocabulary of M terms. On average a document contains K distinct terms, with K much smaller than M, and a typical query contains about L terms, with L much smaller than K (five query terms is a common figure).

The most direct storage scheme reserves one entry for every term in every document: $N \cdot M$ entries in total. In the set-of-words model each entry is a single bit that records whether the term occurs; the bag-of-words model stores a term frequency instead. Almost all of these entries are zero, because each document uses only a small fraction K/M of the vocabulary. Storing the full matrix is therefore wasteful, and a sparse layout that keeps only the K non-zero entries per document reduces storage from $N \cdot M$ to $N \cdot K$ entries.

The sparse layout saves space, but it does not save time. To answer a query we still scan every document, and for each one we read terms that the query never asked about. Here we rely on a property of the classical models: Boolean retrieval, the vector space model, and BM25 all score a document from its query terms alone, under the assumption that terms contribute independently. Only the entries for the query terms affect the result, so almost everything we read is discarded. This is exactly the property the inverted index will exploit.

This property is what makes classical retrieval fast, and it is also its limit. It does not hold for dense retrieval, covered in Semantic Search, where a document’s relevance depends on its whole embedding rather than on the presence of individual query terms. A dense system cannot restrict its attention to a handful of postings, which is why classical lexical retrieval remains orders of magnitude faster than dense retrieval and why the two are often combined.

Inverting the layout

Since only the query terms matter, we can organize storage around terms instead of documents. Rather than storing, for each document, the list of terms it contains, we store, for each term, the list of documents that contain it. This is the inverted index, also called the inverted file. Each term points to a **postings list**: the document identifiers in which the term appears. Figure 4.1 shows the inverted index for the three-document collection above.

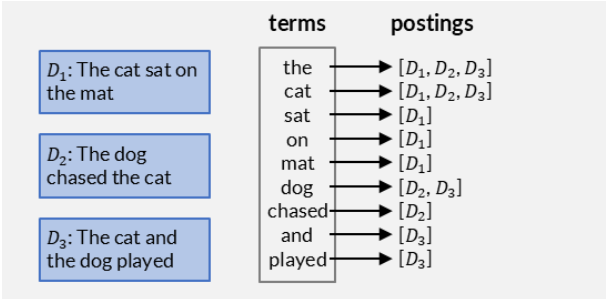

Figure 4.1: An inverted index over a three-document collection: each term in the vocabulary points to its postings list.

Inverting the layout does not change how much we store. There are still $N \cdot K$ term-document pairs. What changes is how much we read per query. A query touches only the postings lists of its query terms. If the $N \cdot K$ pairs are spread over M terms, an average postings list holds $N \cdot K/M$ entries, so a query of L terms reads about $N \cdot K \cdot L/M$ entries instead of all $N \cdot K$.

The read cost is proportional to $N \cdot K$, so inverted files scale linearly with the collection: double the documents and both the index and the average read roughly double. For small to large collections this is comfortable. The book and code indexes, under a gigabyte and around a gigabyte and a half, fit entirely in memory on one machine, so a query with a few keywords is answered almost instantly. The average-term assumption behind those reads often overstates the cost here, because someone searching a book catalog is usually after a specific title and types rare, discriminative words whose postings lists are shorter than average. Under practical conditions, inverted files work very well across this whole range.

The trouble is common words. Stop words and other high-frequency terms have postings lists far longer than average, and they are what makes large collections expensive. On the book and code collections they are a nuisance rather than a blocker: we can drop stop words from the query, searching “Lord of the Rings” as “lord” and “ring” while ignoring “of” and “the”, and lean on the rare, discriminative terms. At web scale the same query behaves differently. A stop word on half of 10 billion pages carries roughly 19 GB of postings on its own, and even after dropping it, “lord” and “ring” each occur in hundreds of millions of pages, some close to 1 GB of postings apiece. No amount of stop-word removal rescues a plain postings scan when the discriminative terms are themselves frequent.

The web index also no longer fits on one machine at more than 20 TB, so it lives mostly on disk and must be distributed. Inverted files degrade gracefully rather than suddenly, remaining practical across a wide range of sizes, but the combination of an enormous collection and unavoidably frequent terms is what eventually breaks the simple scheme. Keeping web-scale queries fast despite gigabyte postings lists is the job of the additional strategies we return to later: skip pointers and early termination, pruned and tiered indexes, caching, the sharding, and scaling out.

Query length is a cost lever in the same way, which matters for query expansion. Expanding a query, introduced in Advanced Text Processing, adds related terms to raise recall, but every added term is another postings list to read and merge, and the extra terms add noise that can lower precision. On a large collection, where a single frequent term can already cost gigabytes, expansion has to be weighed against query cost, not against precision alone.

The structure of the index

An inverted index has three parts. The **vocabulary** (or dictionary) holds the M distinct terms and serves as the lookup key. Each term points to its **postings list**. A separate **document table** stores per-document metadata, such as a title, author, or URL, that the ranker and the result page need but that is not itself searched term by term.

The two tables below make this concrete, using book titles from the library collection we return to throughout the chapter. The document table records one row per document, keyed by an integer identifier:

ID	Title	Author	Year
1	Database Systems: The Complete Book	Garcia-Molina, Ullman, Widom	2008
2	Pattern Recognition and Machine Learning	Christopher Bishop	2006
3	Data Science from Scratch	Joel Grus	2015
4	Operating System Concepts	Silberschatz, Galvin, Gagne	2018
5	Artificial Intelligence: A Modern Approach	Russell, Norvig	2020
…	…	…	…
50	Frankenstein	Mary Shelley	1818

The inverted index maps each term in those titles to the sorted list of document identifiers where it occurs:

Term	Postings (document IDs)
`computer`	[9, 10, 12, 14]
`design`	[8, 9]
`introduction`	[7, 15]
`software`	[9, 13]
`systems`	[1, 14]
`the`	[1, 7, 8, 9, 13, 16, 19, 20, 26, 28, 31, 32, 35, 38, 43, 47]
…	…

The row for term `the` illustrates the earlier point about common words: even in a 50-title collection, the stop word "the" has a longer postings list than any content term.

For Boolean retrieval over the set-of-words model, the postings lists need only the document identifiers; term frequencies and document frequencies are not required. As documents are added, their identifiers are appended to the postings of the terms they contain. When documents arrive in order, each postings list stays sorted by increasing document identifier. That sorted order is not incidental: it is what makes the evaluation below efficient.

Boolean retrieval over postings

A Boolean query combines terms with `AND`, `OR`, and `NOT`. Because each postings list is a set of document identifiers, the operators map directly onto set operations:

- `expr1 AND expr2`: the intersection of the two postings sets.
- `expr1 OR expr2`: the union of the two postings sets.
- `expr1 AND NOT expr2`: the set difference, the documents of `expr1` that are not in `expr2`.

Nesting `AND` and `OR` combines these operations, and the same rules generalize to more than two operands. The `NOT` operator needs care. We never materialize `NOT expr2` on its own, because its complement can contain almost the whole collection. A pure negation, or an `OR` with a negated operand such as "cat OR NOT dog", would force us to enumerate millions of identifiers. Such a query is also rarely what a user means: "cat OR NOT dog" selects every document except those that contain "dog" but not "cat". We therefore allow `NOT` only inside an `AND` that has at least one non-negated operand, and we apply the negated parts last, as a subtraction from the candidates found so far.

In code this is remarkably direct: an inverted index is a dictionary from terms to postings sets, and the Boolean operators are the language's own set operators. Taking an artifical example collection:

```
cat   = index['cat']    # [4, 5, 12, 13, 14, 15, 20, 22, 30, 34]
dog   = index['dog']    # [1, 3, 4, 6, 9, 10, 13, 21, 22, 23, 29, 30]
horse = index['horse']  # [6, 10, 11, 14]
bird  = index['bird']   # [2, 3, 8, 15, 26, 35, 36]

cat & dog      # cat AND dog       -> [4, 13, 22, 30]
horse | bird   # horse OR bird     -> [2, 3, 6, 8, 10, 11, 14, 15, 26, 35, 36]
cat - dog      # cat AND NOT dog   -> [5, 12, 14, 15, 20, 34]

# Queries nest freely, because each operation returns another set of IDs:
(cat & dog) | ((horse & cat) - bird)   # (cat AND dog) OR (horse AND cat AND NOT
bird)
                                       #   -> [4, 13, 14, 22, 30]
(cat | dog) & (horse | bird)           # (cat OR dog) AND (horse OR bird)
                                       #   -> [3, 6, 10, 14, 15]
(cat | dog) - (horse | bird)           # (cat OR dog) AND NOT (horse OR bird)
                                       #   -> [1, 4, 5, 9, 12, 13, 20, 21, 22, 23,
29, 30, 34]
```

The last two lines show the permitted use of `NOT`: negation applied to the result of an `AND`, never on its own. This set-based version is clear but reads every full postings list into memory. The next step scales it down.

Merging sorted postings as streams

Loading whole postings lists into memory does not scale to lists with millions of entries. Instead we read them as sorted streams and merge them, advancing one entry at a time, much like merging sorted lists. Suppose the postings are:

- cat: 1, 4, 10
- dog: 3, 4, 8, 10, 12

To evaluate "cat AND dog" we look at the head of each stream and always advance the smaller one. When both heads are equal, that document satisfies the `AND`, so we emit it and advance both streams.

cat head	dog head	comparison	action	output
1	3	1 < 3	advance cat	
4	3	4 > 3	advance dog	
4	4	equal	emit, advance both	4
10	8	10 > 8	advance dog	
10	10	equal	emit, advance both	10
exhausted	12	cat empty	stop	

The result is the sorted list 4, 10. The merge stops as soon as one stream is exhausted: once cat runs out, no later document can match, even though dog still has the entry 12.

The other operators follow the same pattern with a different emit rule. For "cat OR dog" we emit the smaller head at each step and advance the stream it came from, producing the union 1, 3, 4, 8, 10, 12. For "cat AND NOT dog" we emit a cat entry only when it is strictly smaller than the current dog head, producing 1. Because every operator consumes sorted streams and emits a sorted stream, the operators compose: the output of one merge feeds directly into the next.

The whole scheme depends on the postings being sorted, and we get that ordering for free as long as we only ever append documents with increasing identifiers: a new document's identifier is larger than any already in a list, so it simply goes at the end. Deletions and updates would break this. Removing a document leaves a gap, and changing one alters its terms, so the tidy append-only order no longer holds without rewriting postings lists in place. Many systems avoid that cost by refusing in-place edits. Lucene, for example, treats its index as append-only: every new document is assigned the next identifier and is never modified afterwards, a deletion only flips a marker in a per-segment list of live documents so that the document is filtered out of results, and an update is a delete-by-term, which removes whatever document matches a chosen key term such as a unique identifier, followed by a fresh insert. Documents flagged as deleted are physically removed only later, in bulk, when segments are merged, which we revisit in Practical Frameworks. This design has a valuable side effect. Because existing postings are effectively static, writers and readers barely need to coordinate: new documents are flushed into new segments while queries keep running against the existing ones, so the postings a reader is scanning never shift underneath it.

From matching to ranking

Boolean evaluation answers a yes-or-no question: does a document satisfy the query? It produces a candidate set, but no order within it. For anything beyond exact filtering we want the best documents first, which means scoring each candidate. That splits retrieval into two stages: a **retriever** that uses the inverted index to gather candidates, and a **ranker** that scores them. The next section shows how the ranked models from Chapter 1, the binary independence model, the vector space model, and BM25, all run over the same postings lists.

Ranked Retrieval over Inverted Files

Boolean retrieval returns an unordered candidate set. The ranked models from Classical Text Retrieval, the binary independence model (BIR), the vector space model, and BM25, go further: they assign each document a score so we can return the best matches first. This section shows that all three run over the same inverted index. What changes is the content of the postings and the function that turns them into a score.

Retriever and ranker

Ranked retrieval separates into two stages, shown in Figure 4.2. The **retriever** uses the inverted index to gather candidates, taking the union of the postings lists of the query terms, the same merge as a Boolean `OR`. The **ranker** then scores each candidate with the model's scoring function and keeps the top results. An optional filter can drop candidates that fail a metadata condition before ranking; we return to filtering in the next section.

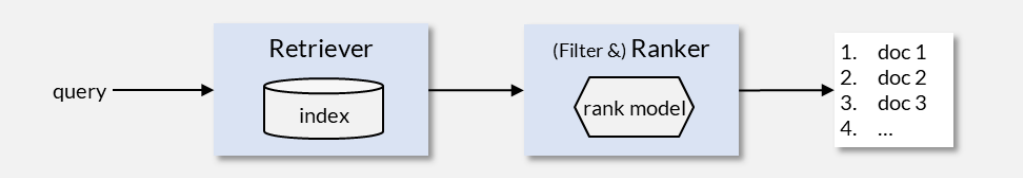

Figure 4.2: The two-stage pipeline: a retriever gathers candidates from the index, and a filter-and-ranker scores them into a ranked list.

The retriever only needs candidates with at least one query term, because a document with no query term scores zero under all three models. This is the same efficiency argument as before: we score a small candidate set, not the whole collection.

Two evaluation strategies

There are two natural orders in which to walk the postings and accumulate scores. We show both using the BIR model because its scoring function, a sum of per-term weights c_j, is additive and involves no term frequencies or document lengths, so the evaluation logic is fully visible without extra lookups.

Term-at-a-time (TAAT)

The most direct extension of the set-based Boolean evaluation from the previous section processes one query term fully before moving to the next. For each term we walk its postings list and add that term's c_j weight to a running score for the document. After all terms are processed, the score dictionary holds every candidate's final score, and we extract the top k.

```
def search_TAAT(query, k):
    query_vector = analyzer.set_of_words(query)
    # filter terms and obtain c_j-weights
    # returns a list of pairs (term_j, c_j)
    term_weights = query_weights(query_vector)

    scores = defaultdict(int)

    # iterate over terms and their postings
    for (term, weight) in term_weights:
        for doc_id in index[term]:
            # add this term's c_j to the document's running score
            scores[doc_id] += weight

    # avoid a full sort: use a heap to extract the top k
    topk = TopKList(k)
    for doc_id, score in scores.items():
        topk.add(doc_id, score)
    return topk
```

TAAT mirrors what we did with Boolean sets: read one term's postings, then the next, then combine. The difference is that instead of intersecting or uniting, we accumulate weights. Its simplicity comes at a cost: the `scores` dictionary can grow very large when a query contains common terms with long postings lists, and we cannot prune early because a document's score is incomplete until the last term is done.

Document-at-a-time (DAAT)

DAAT instead extends the sorted-stream merge from the end of the previous section. Rather than processing one term fully, it advances all query-term streams in parallel and produces one candidate at a time, in document-ID order. Each candidate receives its complete score the moment it appears, so we can feed it directly into a bounded top-k heap.

```
def search_DAAT(query, k):
    query_vector = analyzer.set_of_words(query)
    # filter terms and obtain c_j-weights
    term_weights = query_weights(query_vector)

    # get iterators for each term and fetch the first posting
    iters = [iter(index[term]) for (term, _) in term_weights]
    nexts = [next(it, None) for it in iters]

    topk = TopKList(k)

    while not all(e is None for e in nexts):
        # the smallest doc ID across all stream heads
        smallest = min(nexts, key=lambda x: x or math.inf)

        # score: sum c_j for every term whose head equals smallest
        score = 0
        for j in range(len(nexts)):
            if nexts[j] == smallest:
                score += term_weights[j][1]
        topk.add(smallest, score)

        # advance every stream whose head was the smallest
        for i, e in enumerate(nexts):
            if e == smallest:
                nexts[i] = next(iters[i], None)

    return topk
```

DAAT reads postings as streams and never builds a full score dictionary. Memory is bounded by the heap size k. Because the full score is known the moment a document is emitted, DAAT can also prune: once the heap is full, it can skip candidates whose maximum possible score cannot enter the top k (techniques such as WAND exploit this). The price is slightly more complex bookkeeping in the inner loop.

Comparison

Both strategies read the same postings and consider the same candidates, so their asymptotic cost is similar. TAAT is the clearer extension of the Boolean set approach and a fine choice for short queries. DAAT is generally preferred in production because of its bounded memory and its ability to prune.

The three models over postings

The models differ only in what the postings store and how a candidate is scored.

For **BIR**, the postings need only document identifiers. Each query term t_j carries a weight c_j derived from relevance feedback, and a document's score is the sum of the c_j over the query terms it contains. No term frequencies or document lengths are involved.

For the **vector space model**, the DAAT and TAAT patterns remain the same, but instead of summing a single c_j per query term, we sum the product $d_j \cdot q_j$ for each query term present in the document. Expand-

ing the TF-IDF weights, each term's contribution is $\mathrm{tf}(D, t_j) \cdot \mathrm{tf}(Q, t_j) \cdot \mathrm{idf}(t_j)^2$. As a consequence, the postings for term t_j must store not only the document identifier but also the term frequency $\mathrm{tf}(D, t_j)$ so the ranker can compute that product.

Using the inner product alone (the sum above) already gives a useful ranking that favors documents sharing many weighted terms with the query. The **cosine measure** refines this by normalizing both vectors:

$$\mathrm{sim}_{\cos}(Q, D) = \frac{\sum_j d_j \cdot q_j}{\| \boldsymbol{d} \| \cdot \| \boldsymbol{q} \|}. \tag{4.1}$$

The query norm $\| \boldsymbol{q} \|$ is not a problem: we have the full query vector and can compute it once per query. The document norm $\| \boldsymbol{d} \|$ is the difficulty. During evaluation, whether DAAT or TAAT, we see only the subset of document components that overlap with the query terms, not the full document vector, so we cannot compute the norm from what we read. The solution is to store each document's length (or precomputed norm) in a separate table, indexed by document identifier. This adds one random lookup per candidate document.

BM25 faces the same requirement: its length-normalization factor uses the document length $| D |$ and the average document length avgdl, which likewise must be stored alongside the index. The cost is modest, a single integer or float per document, but it is a structural addition beyond the postings themselves.

For **BM25**, the postings again store term frequencies, and the ranker additionally needs the document length $| D |$ (the total number of term occurrences in D, not the vector norm $\| \boldsymbol{d} \|$), the average document length avgdl, and the parameters k and b.

A worked BM25 example

We index the titles of the 50-book library collection from Performance Evaluation, applying the standard pipeline of tokenization, stop-word removal, and stemming. This gives $N = 50$ documents with an average title length of $\mathrm{avgdl} = 2.72$ tokens. We run the query "database systems", which the pipeline reduces to the stems "databas" and "system", with $k = 1.2$ and $b = 0.75$.

Filtering, Facets, and Compression

Real queries rarely ask for text alone. A user searching a library wants “database systems” among books published after 2000, or filtered to the Computer Science shelf. And a production index must stay small enough to keep its hot data in memory. This section adds two capabilities to the inverted index: metadata filtering, including the faceted navigation built on top of it, and the compression that keeps postings compact.

Filtering on metadata

A predicate is a condition on document metadata, such as `year > 2000` or `genre = "Computer Science"`. There are three ways to combine such a predicate with a text query, differing in when and how the predicate is evaluated.

- **A priori filtering** evaluates the predicate first. We keep document metadata in a structure that answers the condition efficiently, a B-tree or an inverted list on the attribute, and look up the set of document identifiers that satisfy it. That set is passed into the retriever, which drops any candidate not in it during the postings merge. This is the best option when the predicate can be indexed, because it removes non-matching documents before scoring.
- **A posteriori filtering** evaluates the predicate last. When no index exists for the attribute, we rank first and check the predicate only on the documents we are about to return, pulling from the top-k heap in score order and skipping those that fail. For a loose predicate we test only the few documents we return plus a handful skipped. For a highly selective predicate we may have to walk deep into the heap, but that is still cheaper than testing every document in the collection.
- **Inline filtering** evaluates the predicate during the merge. We store the attribute in a stream aligned with the postings and ordered by document identifier, and advance it in lockstep with the postings streams, testing the condition as each candidate appears. This keeps filtering and scoring in a single pass.

In each case the retrieval algorithm itself is unchanged. Filtering slots into the same place where relevance feedback already removes documents: a candidate that fails the predicate is simply not scored or not returned.

Faceted search

Faceted search is filtering turned into navigation. Instead of a free-form predicate, the user picks from categorical attributes, a genre, a publication decade, a content type, and the system both filters the results and reports how many documents fall under each option. This is the a priori approach specialized to categories: the index keeps a separate postings list for each facet value, so filtering by a facet is an intersection with that list, and the facet counts are the list lengths.

Two refinements reduce the storage this adds. **Clustered indexing** groups documents that share facet values, so their postings lists overlap heavily and can be encoded relative to a shared reference list. **Hierarchical facet compression** exploits nested facets: a geographic hierarchy like Country then State then City stores only the differences between levels rather than the full path for every document.

Compressing postings lists

Compression is one of the highest-leverage optimizations in an inverted index. Cutting postings storage by a factor of four or more lets more of the index sit in memory, and reading fewer bytes from disk directly lowers query latency.

The key observation is that a postings list is a sorted sequence of document identifiers. Rather than storing each identifier, we store the gaps between consecutive identifiers, the **d-gaps**. In a list like 95673, 127088, … we store the first identifier and then the gap 31415. Gaps are much smaller than absolute identifiers, especially for frequent terms whose postings are dense, and small integers compress well.

Variable-byte encoding

Variable-byte (VByte) encoding is the most widely used byte-aligned scheme. It uses the most significant bit of each byte as a continuation flag and the remaining seven bits for data. Every byte except the last has its high bit set to 1; the final byte has its high bit set to 0 to mark the end of the number.

Other schemes

VByte is simple and fast, but block-based schemes compress better. **PForDelta** (patched frame-of-reference delta) processes fixed blocks of, typically, 128 d-gaps. For each block it picks the smallest bit width b that fits about 90% of the values and stores those in b bits each; the roughly 10% that do not fit are recorded separately as exceptions. This packs the common case tightly while still handling outliers. Another byte-aligned family uses a two-bit length marker per value (one to four bytes), keeping data byte-aligned for fast decoding and reaching roughly 15 to 20 percent of the uncompressed size.

Metadata alongside postings

Modern indexes store more than document identifiers in a postings list, and the same compression applies to the extra data. **Term frequencies** are written as small integers after each identifier, enabling TF-IDF and BM25 scoring. Because BM25′s saturation function flattens the contribution of high term frequencies, values above a modest ceiling barely affect the score. This means term frequencies can be capped and encoded in just a few bits (four bits suffice in practice) without noticeably hurting retrieval accuracy. **Length-normalization data**, such as document lengths and per-field statistics used by BM25, is precomputed at index time and kept in compressed form, trading a little storage for less work per query. Storing these values next to the postings is what lets the ranker of the previous section avoid a separate lookup for every candidate.

Practical Frameworks

Almost no application builds an inverted index from scratch. The mechanics of the previous sections, postings, merges, ranking, compression, are already implemented, tuned, and battle-tested in libraries and database engines. The engineering choice is which one to adopt, and that choice follows the quality, latency, and storage tradeoff from the chapter opener. This section covers two ready-to-use options: full-text search inside a relational database, and the Lucene library. The distributed engines built on Lucene are the subject of the next section.

Full-text search in your database

If an application already stores its documents in a relational database, the simplest option is often to search them there. PostgreSQL offers full-text search as a built-in feature, and SQLite provides a similar capability through its FTS5 module. There is no separate service to run and no index to keep synchronized with the source data.

PostgreSQL models text search with two data types. A `tsvector` is a document reduced to a sorted set of normalized lexemes, produced by `to_tsvector`, which tokenizes the text, removes stop words, and applies stemming through a configurable dictionary. A `tsquery` is a search expression with Boolean operators and phrase constraints, produced by `to_tsquery`. The match operator `@@` tests whether a `tsvector` satisfies a `tsquery`.

The speed comes from a GIN index, which stands for Generalized Inverted Index and is a genuine inverted index in the sense of this chapter. GIN stores a set of (key, posting list) pairs, where each key is a lexeme and its posting list holds the row identifiers of the documents that contain it. The lexemes themselves are organized in a B-tree so a lookup is fast, and posting lists are stored compressed; a lexeme with very many rows is promoted from a flat posting list to its own B-tree of row identifiers. A multi-word query resolves each lexeme to its posting list and intersects them, the same sorted-postings merge we saw in the first section. To keep inserts cheap, GIN can buffer new entries in a pending list and merge them into the main structure later, trading a little query overhead for faster writes.

Because a `tsvector` column and a GIN index live inside the database, full-text predicates combine naturally with ordinary SQL: a single query can match text, filter on other columns, join related tables, and run inside a transaction. The following example shows the full pattern, from schema to ranked search with a predicate:

```
-- 1. Define a table for movies
CREATE TABLE movies (
  id        SERIAL PRIMARY KEY,
  title     TEXT NOT NULL,
  overview  TEXT NOT NULL,
  rating    REAL,
  year      INT,
  tsv       tsvector
);

-- 2. Populate the tsvector column for full-text search
--    'A' = highest weight (title gets more influence)
--    'B' = lower weight (overview contributes less)
UPDATE movies
SET tsv = setweight(to_tsvector(title), 'A') ||
          setweight(to_tsvector(overview), 'B');

-- Create a GIN index for fast lookup
CREATE INDEX movies_tsv_idx ON movies USING GIN(tsv);

-- 3. Search with ranking and a predicate on year
SELECT id, title, year,
       ts_rank(tsv, to_tsquery('Star & Wars')) AS r
```

```
FROM movies
WHERE tsv @@ to_tsquery('Star & Wars')
  AND year < 2000
ORDER BY r DESC
LIMIT 10;
```

Step 2 builds the inverted index: `to_tsvector` tokenizes, normalizes, and stems the text, and `setweight` assigns one of four weight labels (`A` through `D`) so that `ts_rank` can weight matches differently by field; here title matches (weight `A`) count more than overview matches (weight `B`). Step 3 combines term matching (`@@`), a metadata predicate (`year < 2000`), and frequency-based ranking (`ts_rank`) in a single query. The database evaluates the GIN index lookup and the year filter together and chooses an efficient execution plan.

For collections up to a few million documents on a database you already operate, this is often all the search infrastructure an application needs. Conceptually, one could even build the inverted index by hand as an ordinary table of (term, document, frequency) rows with a B-tree index on the term column; the hands-on notebook does exactly this to make the structure concrete.

Lucene

Apache Lucene, created by Doug Cutting in 1999 and an Apache project since 2001, is the open-source Java library at the core of most production search platforms. It provides the full toolkit of this chapter: analysis, inverted indexing, term weighting, and ranking. Figure 4.3 shows how its pieces fit together. The current major version is Lucene 10 (10.5 as of mid-2026). Major releases change APIs; in particular, Lucene 10 removed the convenience `searcher.doc()` method (use `reader.storedFields()` instead) and requires an explicit `Field.Store` argument when constructing numeric fields. The concepts below are stable across versions; the hands-on notebook uses the Lucene 10 API.

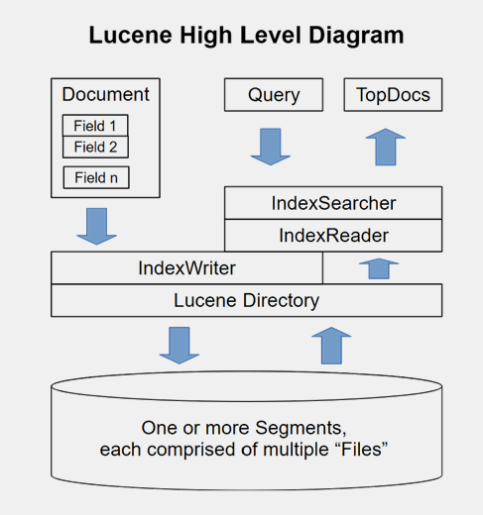

Figure 4.3: Lucene's architecture: the write path (Document, IndexWriter, Directory, Segments) and the read path (Query, IndexSearcher, IndexReader, Directory, Segments, TopDocs) meet at the Directory storage abstraction.

Documents, fields, and analyzers

Lucene models a document as a set of named fields, and not every document needs every field. A field carries a name, a value, and a type that specifies whether the value is tokenized, what is written to the inverted index, and whether the original value is stored for retrieval. These three choices are independent: a long body field is typically tokenized and indexed for search but not stored, so the application fetches the original text from its own database. Field subclasses cover the common cases: a text field is tokenized and indexed with term frequencies and positions; string and numeric fields are indexed without tokenization for exact-match and range queries on metadata; a stored-only field is kept but not searchable.

Fields are indexed independently. If both a title field and a body field contain "house", Lucene treats the two occurrences as distinct terms, internally prefixed as "title:house" and "body:house". This is what lets a search weight a title match more heavily than a body match, or apply different normalization to each field.

Tokenization is the job of an analyzer, and the choice of analyzer must be identical for indexing and for querying. The standard analyzer lowercases and strips punctuation; the English analyzer additionally removes stop words and applies a Porter stemmer; custom analyzers can combine filters such as stemming, length limits, and case handling. Changing the analyzer means rebuilding the index.

```
// Comparing analyzer output for the same text
var text = "I think text's values' color goes here; WHAT happens ...";

// StandardAnalyzer: lowercase, strip punctuation, keep possessives
print_tokens(new StandardAnalyzer(), text);
// -> i think text's values color goes here what happens ...

// EnglishAnalyzer: + stop-word removal + Porter stemming
print_tokens(new EnglishAnalyzer(), text);
// -> think text valu color goe here what happen ...

// Custom stop words
var stopWords = new CharArraySet(Arrays.asList("i", "do"), false);
print_tokens(new EnglishAnalyzer(stopWords), text);
// -> think text valu color goe here what happen ...
```

Segments and updates

Lucene writes the index as a set of immutable **segments**. Each time an `IndexWriter` flushes, it creates a new segment, which reduces contention and protects against corruption. Because segments are immutable, Lucene never edits a document in place, and it has no primary key: internally a document is known only by a `docId` that can change when segments merge. Deletes and updates therefore identify documents by content. A **delete-by-term** marks every document whose chosen key field holds a given value, for example `id:"B-10432"`, by flipping a bit in the segment's live-documents list rather than erasing any data; a **delete-by-query** does the same for documents matching an arbitrary query. An update is a delete-by-term of the key value followed by an add of the new version, applied atomically. A configurable merge policy periodically combines smaller segments into larger ones, and only during a merge are the documents flagged as deleted physically dropped. Searching runs over all live segments in parallel and merges their results, which is the seed of the distributed scaling in the next section.

The following shows how documents are built and added to an index:

```
// Set up analyzer, directory, and writer
Analyzer analyzer = new EnglishAnalyzer();
Directory directory = FSDirectory.open(Paths.get("./index"));
IndexWriter writer = new IndexWriter(directory, new IndexWriterConfig(analyzer));

// Build a document from application data
Document doc = new Document();
doc.add(new TextField("title", "Star Wars", Field.Store.YES));
doc.add(new IntField("year", 1977, Field.Store.YES));
doc.add(new TextField("body", "A long time ago ...", Field.Store.NO));
writer.addDocument(doc);

// Close the writer to flush a new segment
writer.close();
```

Each `TextField` is tokenized and indexed; the `IntField` supports exact and range queries on metadata. Setting `Store.NO` on the body means its text is searchable but not retrievable from results, keeping the stored-fields footprint small.

Querying and scoring

A search uses an `IndexSearcher` over the index directory, usually with a query parser that turns user input into a query and applies the same analyzer used at indexing time. Results come back as a `TopDocs` object

holding the best-scoring documents and their scores. Beyond plain keyword queries, Lucene supports field-scoped terms with boosts (a title match weighted higher), range queries, wildcards, and fuzzy matching within an edit distance. Queries can also be built programmatically, combining clauses that must, should, or must-not match, or that only filter without affecting the score.

Lucene ranks with BM25 by default, computed per field so that the document frequency and length normalization use that field's statistics. Each query clause contributes a per-term score, and the document's final score is their sum:

$$\begin{aligned} \text{score}(D, Q) &= \sum_i \text{score}_i \\ \text{score}_i &= \text{boost}_i \cdot \text{idf}_i \cdot \frac{\text{tf}_i \,(k_1 + 1)}{\text{tf}_i + k_1\left(1 - b + b\,\frac{dl}{avgdl}\right)} \\ \text{idf}_i &= \log\left(1 + \frac{N - n_i + 0.5}{n_i + 0.5}\right) \end{aligned} \tag{4.2}$$

Here score_i is the contribution of one query term (or clause), and clauses marked as filters contribute zero.

This is the BM25 of the previous section with two practical additions: a per-clause `boost` factor, and the Lucene IDF variant with the extra 1 inside the logarithm that keeps the weight positive even for very common terms. A helpful diagnostic, `explain`, prints how each clause contributed to a document's score, which is invaluable when tuning boosts and field weights.

```
// Set up a searcher and a multi-field query parser
IndexSearcher searcher = new IndexSearcher(DirectoryReader.open(directory));
QueryParser parser = new MultiFieldQueryParser(
    new String[]{"title", "body"}, analyzer);

// Simple keyword query (searches title and body, ranked by BM25)
Query q = parser.parse("star wars");
TopDocs results = searcher.search(q, 10);

// Access stored fields (Lucene 10 API)
StoredFields storedFields = searcher.getIndexReader().storedFields();
for (ScoreDoc hit : results.scoreDocs) {
    Document d = storedFields.document(hit.doc);
    System.out.printf("%.2f  %s (%s)\n", hit.score,
        d.get("title"), d.get("year"));
}

// Programmatic query with filter, boost, and explain
Query query = new BooleanQuery.Builder()
    .add(IntField.newRangeQuery("year", 1950, 2000), Occur.FILTER)
    .add(new TermQuery(new Term("title", "shawshank")), Occur.MUST)
    .add(new BoostQuery(
        new TermQuery(new Term("body", "hope")), 1.5f), Occur.SHOULD)
    .build();

TopDocs top = searcher.search(query, 10);
// Score breakdown for the top hit
System.out.println(searcher.explain(query, top.scoreDocs[0].doc));
```

The `explain` output decomposes the score into each clause's contribution, showing the IDF, TF, field length, boost, and normalization parameters. For the query above, it might look like this:

```
7.4038 = sum of:
  0.0 = match on required clause, product of:
    0.0 = # clause
    1.0 = year:[1950 TO 2000]           // FILTER: must match, no score
```

```
1.0 = year:[1990 TO 2000]             // SHOULD: contributes 1.0
3.0980 = weight(title:shawshank in 0) [BM25Similarity], result of:
  3.0980 = score(freq=1.0), computed as boost * idf * tf from:
    6.5023 = idf, computed as log(1 + (N - n + 0.5) / (n + 0.5)) from:
      1 = n, number of documents containing term
      999 = N, total number of documents with field
    0.4765 = tf, computed as freq / (freq + k1 * (1 - b + b * dl / avgdl)) from:
      1.0 = freq, occurrences of term within document
      1.2 = k1, term saturation parameter
      0.75 = b, length normalization parameter
      2.0 = dl, length of field
      2.2533 = avgdl, average length of field
3.3057 = weight(body:hope in 0) [BM25Similarity], result of:
  3.3057 = score(freq=1.0), computed as boost * idf * tf from:
    1.5 = boost                       // the BoostQuery factor
    4.7687 = idf, computed as log(1 + (N - n + 0.5) / (n + 0.5)) from:
      8 = n, number of documents containing term
      1000 = N, total number of documents with field
    0.4621 = tf, computed as freq / (freq + k1 * (1 - b + b * dl / avgdl)) from:
      1.0 = freq, occurrences of term within document
      1.2 = k1, term saturation parameter
      0.75 = b, length normalization parameter
      8.0 = dl, length of field
      8.335 = avgdl, average length of field
```

Every quantity from the BM25 formula is visible: the IDF with its `log(1 + ...)` variant, the saturating TF with `k1`, `b`, `dl`, and `avgdl`, and the boost factor. Filter clauses contribute zero to the score but must still be satisfied. This makes it possible to diagnose why a document ranks where it does and to tune field boosts accordingly.

Higher-level pipelines

Above these engines sit frameworks that orchestrate retrieval as one step in a larger pipeline, combining keyword search with dense retrieval, rerankers, and language-model generation. Haystack is a widely used example. Because these frameworks belong to the retrieval-augmented generation setting, we cover them in Retrieval-Augmented Generation rather than here.

Scaling Out

A single Lucene index scales a long way. It handles up to about 2.1 billion documents, and its segment structure lets one query run in parallel across segments. But a single node has limits. Once the index grows past roughly 20 to 40 GB, query latency is bound by memory and disk throughput, and a single machine cannot serve hundreds of concurrent queries. Beyond that point we distribute the index across many machines. This section works up from parallelism inside one node to a globally distributed deployment, using the three engines built on Lucene: Apache Solr, Elasticsearch, and OpenSearch.

The distributed engines

All three engines wrap Lucene and add clustering, horizontal scaling, and near-real-time updates. **Solr** offers a schema-driven, configuration-centric model with strong support for faceting and multilingual text, and is a common choice for enterprise and site search. **Elasticsearch** exposes a schema-flexible REST API and is heavily used for log and security analytics, where together with Logstash and Kibana it forms the ELK stack, though it is equally a capable document search engine. **OpenSearch** began in 2021 as an Apache 2.0 fork of Elasticsearch, created after Elastic changed the Elasticsearch license, and is led by a community with Amazon's backing while staying largely compatible with Elasticsearch.

Elasticsearch and OpenSearch matter beyond keyword search: both have added dense vector (approximate nearest-neighbor) search, so the same cluster can serve lexical and semantic retrieval together. We return to that hybrid setup when we cover vector search in a later chapter.

Parallelism within a node

The first level of scaling needs no extra machines. Lucene can split an index into several segments, and a merge policy controls how many there are. Figure 4.4 shows the pattern: suppose we keep 15 equal segments on a server with 16 virtual CPUs. A coordinator thread parses the query and hands each segment to a searcher thread on one of the remaining 15 CPUs. Each thread searches its segment, and the coordinator merges the partial results before replying. This is a scatter-gather, or map-reduce, pattern: map the query across segments, reduce the partial results into one ranked list.

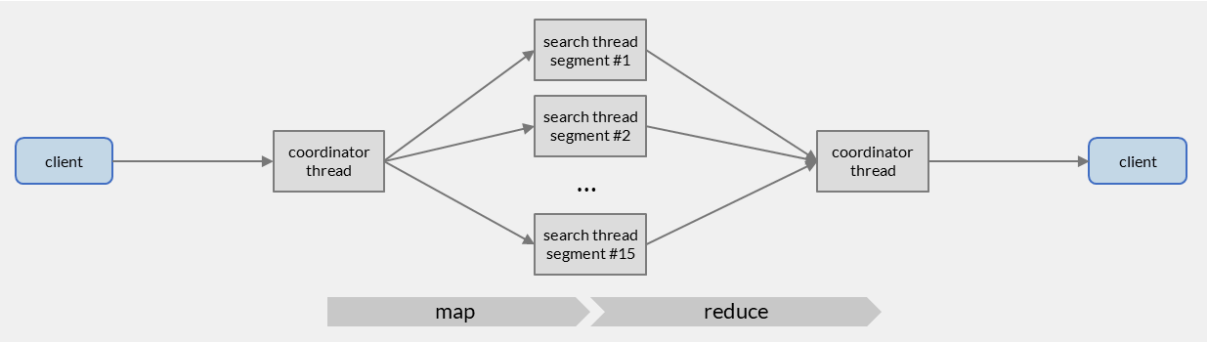

Figure 4.4: Segment-level parallelism: a coordinator thread scatters a query across per-segment searcher threads and gathers their partial results.

The speedup is real but bounded. If 99% of the work is the segment search, Amdahl's law gives roughly a 13-fold speedup over a single thread. But latency still grows with the index size, only one query runs at a time at full width, and pushing to ever-larger machines produces a monolithic server with no high availability. To go further we need more nodes, not more cores.

Sharding across nodes

Sharding distributes the collection across many smaller worker nodes. Each **shard** holds a distinct subset of the documents as its own Lucene index, with its own segments and term statistics. Search now uses two levels of map-reduce, shown in Figure 4.5: the query is scattered across shards, and within each shard across its segments, then reduced back up.

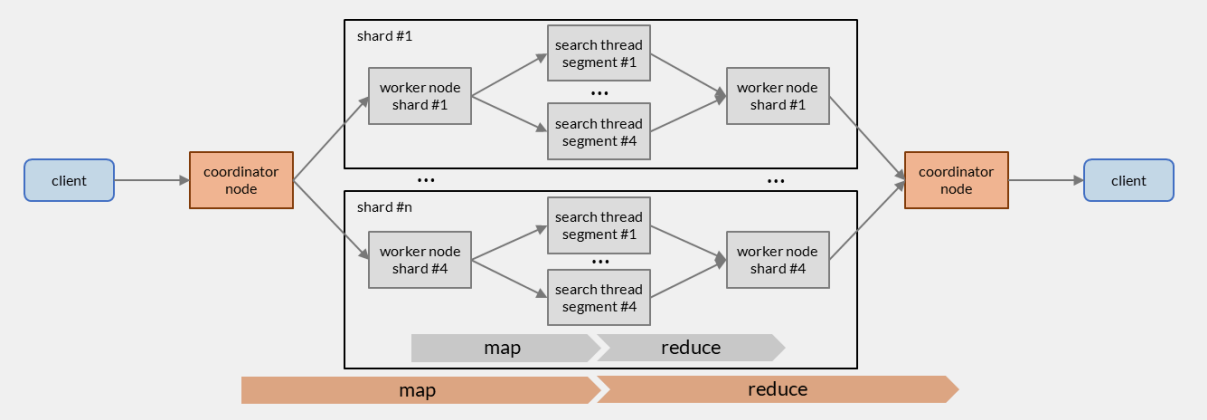

Figure 4.5: Two-level map-reduce: a coordinator fans the query out across shards, and each shard fans it out across its segments before results merge bottom-up.

Because each shard keeps its own term statistics, the same document could score slightly differently depending on which shard holds it. Search is not an exact operation, so small score differences across shards are acceptable as long as they do not distort the overall ranking. When a document is added, a coordinator node assigns it to a shard, either by a policy such as round-robin or by hashing a chosen prefix of the document identifier so that related documents land on the same shard. The shard count is usually fixed, because redistributing documents is expensive. With n shards, the system can handle up to n times the single-node document limit (about 2.1 billion documents), so adding shards is what lets a collection grow beyond what one machine supports. Each shard runs on a modest worker node, for example a container in a Kubernetes cluster, rather than requiring one ever-larger server.

Replication and availability

Sharding spreads data out but does not by itself protect against node failure. For that we replicate each shard and place the replicas in different availability zones, avoiding the same server, rack, or data center. Figure 4.6 shows a deployment of four shards, each with one leader and one replica, arranged so that a shard's leader and replica sit in different zones. Within a shard, the leader node indexes new documents and propagates the changes to its replicas; the engines replicate at the storage level so every replica returns identical results.

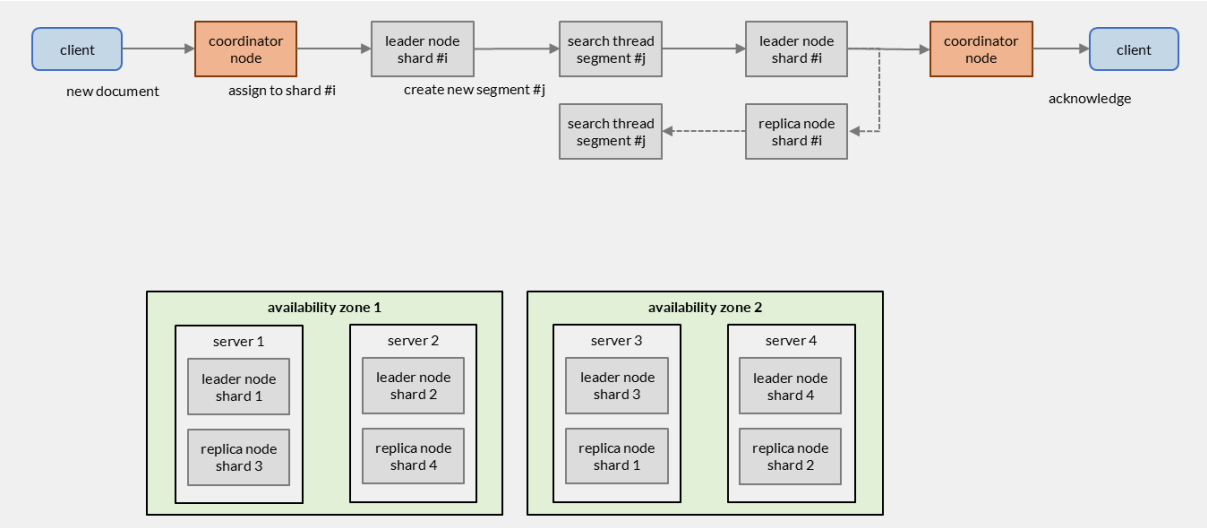

Figure 4.6: Leader and replica shards placed across two availability zones: new documents are indexed by the leader and propagated to the replica, and each shard's replica sits in a different zone from its leader.

Replicas do more than protect against failure. Each leader can also act as a coordinator, routing each shard's part of a query to any replica or leader that holds it. Adding replicas therefore adds concurrent-query capacity roughly in proportion to their number, up to the capacity of the coordinating nodes. Scaling

concurrency this way needs more physical servers, but they can be modest ones hosting small container nodes.

Distributing across regions

The highest level of distribution places whole clusters in several geographic regions near the users they serve. Figure 4.7 shows two regions, one for Europe and one for Asia. Writes go to a primary region and are replicated across regions; a DNS service with a geoproximity policy resolves the application hostname to the entry point of the nearest region, with the other region as a fallback. A client is routed to its closest cluster, cutting the round-trip latency that a distant single region would impose.

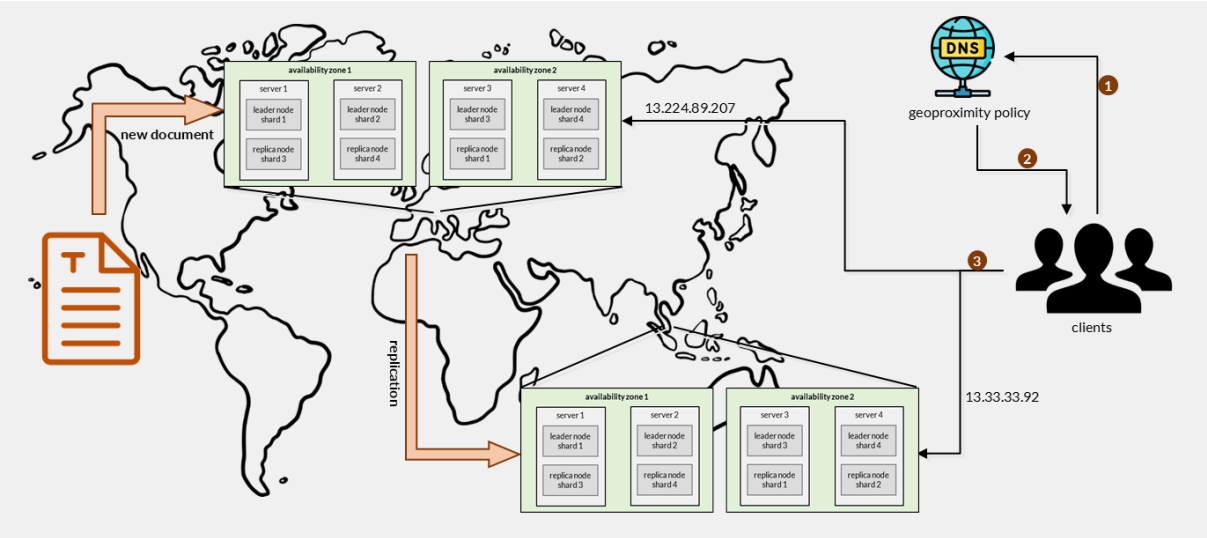

Figure 4.7: Geo-distributed clusters: writes replicate from a primary region to another, and DNS geoproximity routing sends each client to its nearest cluster.

Regional distribution does not speed up a single query. With inter-region latencies of 100 to 200 ms, splitting one search across regions would cost more than it saves, so each query runs within one region. What regional distribution buys is more total concurrent capacity, higher availability, and lower latency for clients near a region. Without it, clients far from a single cluster would see every query pay the long round trip.

Summary

Approach Comparison

The chapter surveyed three ways to run inverted-file search in practice. They sit at different points on the quality, latency, and storage tradeoff.

Property	In-database FTS	Lucene library	Distributed engines
Examples	PostgreSQL, SQLite FTS5	Apache Lucene	Solr, Elasticsearch, OpenSearch
Scale	up to a few million documents	one node, up to ∼2.1B docs or 20-40 GB before latency-bound	10s of billions, sharded and replicated
Default ranking	frequency and proximity, no BM25	BM25	BM25, plus vector search
Vector search	via pgvector extension	Lucene 9+ (HNSW)	native, with hybrid scoring
Extra operations	none, it is your database	embed in a JVM application	run and operate a cluster
Best fit	an app that already has a database	custom search inside one application	high query volume, high availability, hybrid retrieval

Each tier builds on the same core: postings, merges, and BM25 scoring are present everywhere. What grows from one tier to the next is the surrounding machinery for scale and availability.

Key Takeaways

1. Scoring every document per query does not scale. The inverted index maps each term to a postings list, so query cost grows with the number of query terms, not with the size of the collection.
2. Retrieval splits into a retriever that gathers candidates by merging query-term postings and a ranker that scores them. Boolean, BIR, vector space, and BM25 all run over the same index; only the postings content and scoring function differ.
3. Document-at-a-time evaluation keeps memory bounded and allows pruning; term-at-a-time is simpler but accumulates a potentially large score table and cannot prune early.
4. Metadata predicates attach through a priori, a posteriori, or inline filtering, and faceted search is the a priori case specialized to categories. Compression via d-gaps and variable-byte encoding keeps postings small so more of the index stays in memory.
5. Sharding scales data volume and per-query latency; replication scales concurrent-query capacity and availability. A single query is not split across regions, so geo-distribution buys capacity, availability, and proximity, not lower per-query latency.

Key Formulas

Self-Check Questions

1. (Understand) Explain why an inverted index reads only a fraction L/M of the data a full scan would touch, and what L and M stand for.
2. (Understand) Given the postings cat = 1, 4, 10 and dog = 3, 4, 8, 10, 12, evaluate “cat AND dog”, “cat OR dog”, and “cat AND NOT dog” by streaming merge.
3. (Analyze) When would you prefer term-at-a-time over document-at-a-time evaluation, and when would you choose a posteriori filtering over a priori filtering?
4. (Analyze) Why can the same document receive slightly different scores on different shards, and why is that usually acceptable?
5. (Evaluate) An application with two million documents already stored in PostgreSQL needs search with good relevance ranking. Weigh in-database full-text search against adding a Lucene-based engine, using the quality, latency, and cost tradeoff.

Further Reading

- Manning, C. D., Raghavan, P., & Schütze, H. (2008). **Introduction to Information Retrieval**. *Cambridge University Press*. Free HTML. The chapters on index construction and index compression cover this chapter's core structures in depth.
- Zobel, J., & Moffat, A. (2006). **Inverted Files for Text Search Engines**. *ACM Computing Surveys*, 38(2). The canonical survey of inverted-file representation, construction, and query evaluation, including compression and query-time optimizations.
- Robertson, S., & Zaragoza, H. (2009). **The Probabilistic Relevance Framework: BM25 and Beyond**. *Foundations and Trends in Information Retrieval*, 3(4), 333-389. PDF. Derives the BM25 ranker used throughout the chapter, including the multi-field BM25F variant behind Lucene's per-field scoring.
- Büttcher, S., Clarke, C. L. A., & Cormack, G. V. (2010). **Information Retrieval: Implementing and Evaluating Search Engines**. *MIT Press*. An implementation-focused treatment of indexing, compression, and query processing.

Tools and documentation

- Apache Lucene: lucene.apache.org
- Apache Solr: solr.apache.org
- Elasticsearch: elastic.co
- OpenSearch: opensearch.org
- PostgreSQL Full Text Search: postgresql.org
- Zhou, Z. (2019). **Lucene IndexWriter: An In-Depth Introduction**. *Alibaba Cloud Blog*. Article. Walks through IndexWriter internals: the concurrency model, segment creation, delete-by-term, flush, commit, and merge lifecycle.